

Python Backend Assignment

Objective

Build a generic IoT Data Ingestion & Streaming Service.

The system allows:

- Creating and managing users
- Ingesting IoT data for a user
- Fetching historical & latest data
- Real-time updates via WebSocket

All APIs and WebSockets must be authenticated.

Tech Stack (Required)

- Python 3.10+
- FastAPI (preferred) or Django REST Framework
- OpenAPI or Postman
- Async support
- MongoDB : <https://www.mongodb.com/pricing> (Use free tier)
- WebSockets
- JWT-based authentication
- Git

Other:

- Docker
- Redis
- Pytest

Core Concepts

- API design
- Async programming

- WebSocket auth & broadcasting
- Data validation rules
- MongoDB schema design
- Clean architecture

Functional Requirements

Authentication

Login

POST /auth/login

Payload

```
{  
  "username": "admin",  
  "password": "password"  
}
```

Response:

```
{  
  "access_token": "JWT_TOKEN"  
}
```

Token must be required for all REST APIs and WebSocket connections, except signin/signup.

User Management APIs

Create User

POST /users

```
{
  "user_id": "U1001",
  "name": "Test User",
  "status": "active"
}
```

Update User

PUT /users/{user_id}

```
{
  "name": "Updated Name",
  "status": "inactive"
}
```

Get User

GET /users/{user_id}

IoT Data Model (Generic)

Each data point contains:

```
{
  "user_id": "U1001",
  "metric_1": 34.5,
  "metric_2": 78,
  "metric_3": 1,
  "timestamp": 1769535468
}
```

Ingest IoT Data

a) Ingest via REST API

POST /iot/data

Rules (Example – candidate may extend):

- metric_1: 0 – 100
- metric_2: 0 – 200
- timestamp must not be in future
- user must exist & be active

Reject invalid data with proper error messages.

b) Ingest via WebSocket

WS /ws/ingest

- JWT token passed via headers or query param
- Incoming messages follow same schema & rules
- Valid data must be:
 - stored in MongoDB
 - forwarded to subscribers

Fetch IoT Data

a) REST API

Latest Data

GET /users/{user_id}/iot/latest

Historical Data

GET /users/{user_id}/iot/history?limit=50

b) WebSocket Subscription

WS /ws/subscribe?user_id=U1001

Behavior:

- Client subscribes to a user's data stream
- Whenever new IoT data arrives (via API or WS):
 - push real-time message to subscribers

Example WS message:

```
{
  "event": "NEW_DATA",
  "data": {
    "metric_1": 45.2,
    "metric_2": 88,
    "metric_3": 0,
    "timestamp": 1710000100
  }
}
```

WebSocket Authentication (Mandatory)

- JWT must be validated during connection
- Unauthorized connections must be rejected
- Token expiry should disconnect the client

Other (Pick Any One)

- Dockerize the service
- Add Redis pub/sub for WebSocket scaling
- Add unit tests
- Add rate limiting

Submission Guidelines

- Proper error handling
- GitHub repository
- README must include:
 - Setup instructions
 - Auth flow explanation
 - API & WebSocket examples
 - Design decisions